Trezo Rebuild Blueprint

The rules, formulas, architecture, and prerequisites of the Trezo trading platform — everything needed to recreate it in a clean environment. Extracted from the live codebase at commit 4163078 (main), 2026-09-08. No keys or secrets appear anywhere in this document, by design: every credential is listed by NAME so you know what to obtain, never by value.

How to use this document. Section 1–3 describe what the platform is and how a trade flows through it. Section 4 is the heart: every trading rule with its actual formula and the numbers running in production today. Section 5 is the safety core — the rules that exist because real incidents happened; build these FIRST in any rebuild. Sections 6–7 cover alerting and operations. Section 8 is your rebuild checklist (accounts, keys by name, env vars, database, bring-up order). Section 9 is the lessons list — the noise you want to design out rather than carry over.

The code itself travels beside this PDF as a zip of the repository at the same commit (no .env files, no keys — they are gitignored and were never committed). Your GitHub repo aynvmike/trezo-platform remains the canonical copy; the zip is the same tree, portable.

1. What Trezo is

Trezo is an agentic paper-trading platform: one Python process runs 30 cooperating agents that scan markets, argue about risk, place orders at Alpaca (paper), and watch every open position around the clock. A Next.js dashboard shows the books and exposes Bot Tuning (per-book settings). Supabase (hosted Postgres) is the single source of durable state.

Piece	Runs as	What it is
Engine (agents)	TrezoAgents service, port 8001	Python 3.11+ / FastAPI + APScheduler. One process, one event loop, 30 agents on an in-process r
API	TrezoApi service, port 8000	FastAPI; serves the web app's data needs.
Web	TrezoWeb service, port 3000	Next.js dashboard + Bot Tuning. (You are replacing this with a new site; the engine does not depen
Database	Supabase project	Postgres + RLS. Ledger, settings, activity log, ops relay mailbox, briefings.
Broker	Alpaca paper API	Three paper books under one login: primary (~5k), 25k, 75k. Positions, orders, fills, account activitie
Host	Windows VM + nssm	Three nssm services. Any OS works; nssm is just the Windows service wrapper.

Hard operating rule: never two engines on one Alpaca account. The old engine stops before the new one starts; cutover is a human decision, not an automatic one.

2. The 30 agents

Group	Agents	Job
Scanners	pattern_detection, stms_scanner, orb_scanner, exte[illegible]	[illegible] Each lane
Deciders	risk_manager, portfolio_architect, adaptive_scope, cy[illegible]	[illegible]ches with a spoken veto.
Executors	trade_execution	Fans an approval out per book, sizes it, places the order, writes the ledger row.
Monitors	position_monitor, exit_advisor, exit_advisor_options, [illegible]	[illegible] rules, gap handling; reconcile ledger vs broker; per-book h
Income	dividend_manager, tax_optimizer, kindrip	Dividend ladder lane, tax-aware accounting, drip handling.
Ops/support	ops_watchdog, market_desk, archivist, relay_ingest, [illegible]	[illegible] alarms + ops relay), market reports reader, hourly a

Agents communicate only via the bus (AgentMessage: kind = signal / approve / veto / execute / error / info, plus a payload). The watchdog counts these kinds to detect a silent pipeline (Section 6). Every agent has a tick interval; the

scheduler enforces per-agent tick ceilings so one slow agent cannot starve the loop.

3. The decision pipeline (life of a trade)

```
scanner signal (ticker, side, TCS, stop%, target%)
   -> risk_manager gates:  kill-switch -> direction -> liquidity -> crowding
                           -> variance premium (options) -> TCS bar -> R:R harmonizer
   -> APPROVE (or spoken veto)
   -> trade_execution, PER BOOK:  already-holds? -> book cap? -> pocket cap?
                                  -> sizing.plan_position (Section 4.1-4.2)
                                  -> order to Alpaca -> ledger row
   -> position_monitor every 60s: stop / target / trail / time rules / gaps
   -> reconciler + adoption + trade QA: ledger must equal broker, always
```

Two invariants shape everything: **(1)** every deliberate refusal anywhere in this pipe must be audible (an activity-log row naming the book and the reason), and **(2)** every read that can fail must be distinguishable from an empty answer (Section 5).

4. Rules and formulas (with production values)

4.1 Position sizing

```
risk_usd   = equity x risk_pct             # risk_pct: Bot Tuning slider, default 0.01 (1%)
stop_dist  = |entry - stop|
risk_qty   = risk_usd / stop_dist
cap_pct    = book's max_position_pct (Bot Tuning) else 0.25
notional_cap = min(equity x cap_pct, buying_power)
qty        = min(risk_qty, notional_cap / entry)   # stocks: whole shares only
```

Sizing reads the settings row OF THE BOOK BEING SIZED (plan_position takes user_id) — never an ambient/global row. This is load-bearing; see lesson L1.

4.2 Reward:risk floor (the R:R gate)

```
reward_risk = round((target - entry) / stop_dist, 2)
floor       = this book's min_reward_risk   # seed 1.5 if unset; clamp [0.3, 3.0]
REJECT if reward_risk < floor - 0.011       # one 2-decimal rounding step of grace
```

The Risk Manager's harmonizer SHAPES each approval to land exactly at the floor: **stop_pct = target_pct / floor**, computed per book (event rr_reharmonized names each book's floor and the reshaped stop). Because trades arrive exactly AT the floor, the gate and the harmonizer must read the SAME per-book value — a mismatch rejects 100% of trades forever (this outage happened; the 0.011 tolerance also absorbs cent-rounding of real prices). Current books run floors of 0.4 (primary) and 0.5 (25k/75k); targets have a floor of TREZO_TARGET_MIN_PCT = 0.006 (0.6%).

4.3 The confidence bar (TCS)

```
effective_min_tcs = min_tcs + scope_bump + cycle_bump + margin_bump + recovery_bump
APPROVE only if signal TCS >= effective_min_tcs
```

- min_tcs = the book's tcs_threshold (Bot Tuning); for signals shared across books, the FLOOR across books is used so one strict book cannot silence the lane for the others.
- Bumps: adaptive_scope (recent performance), cycle_awareness (market regime), margin usage, and kill-switch recovery (+bump while a book earns its way back). Coverage mode caps the bar at 40 so a starved lane can still trade small. Crypto has its own TCS floor of 35.
- ORDERING RULE: the kill-switch block that ASSIGNS recovery_bump must run before the sum that READS it. Violating this crashed every directional signal for four trading days, silently (lesson L2). Guard tests pin the order by AST inspection.

4.4 Books, pockets, and capacity

```
book_cap   = max_open_positions per book (default 14)
pocket_cap = max(1, round(book_cap x alloc_share / total))   # per market type
refusals   : book_already_holds | book_at_capacity | pocket_at_capacity
```

Each book divides its slots into POCKETS by market type (stock / crypto / options / dividend) according to its posture ('auto' lets the allocator decide; posture also sets dollar budgets per pocket, e.g. the 75k book's stocks budget). A full pocket refuses new names so other pockets keep their chairs; a priority-rotation can ask the weakest hold to leave when demand queues. Every refusal writes an activity row with book, pocket, and open/cap numbers (lesson L4: a refusal that says nothing is a silent trade-dropper).

4.5 Exit rules — stocks and scalps

Rule	Trigger	Value
Stop / target	always	The harmonized stop and target from approval; bracket at broker where supported.

Rule	Trigger	Value
Profit trail	gain >= arm threshold	Stop ratchets up to lock (1 - giveback) of the gain; sells on giveback. Scalps arm earlier than swings
Max hold (scalp)	held >= 90 min	MAX_HOLD_MINUTES = 90; intraday strategies only.
Stagnation	held >= 75 min and < 0.25R	STAGNATION_MINUTES = 75, STAGNATION_R = 0.25.
End of day	3:45 PM ET	force_exit_345pm for any intraday strategy. Calendar rule, deliberately NOT shielded. (Hardcoded 1
Gap handling	open-bell gap through a level	Gap down through stop: close at market (cap bleed). Gap up: trail ratchets the lock. Audit row says
Adopted-row shield	row adopted with backdated entry_at	Time rules do not market-sell a row on its first tick just because its true entry is old; shield expires o

4.6 Exit rules — crypto (24/7 lane)

- No native brackets at Alpaca crypto: the engine keeps a RESTING stop order at the broker and ratchets it (broker_stop_armed). The stop's quantity is the VENUE'S OWN position quantity, never the ledger's rounding of it (lesson L3).
- MAE ceiling: a losing coin past its maximum-adverse-excursion ceiling is closed; for adopted rows this is gated behind TREZO_CRYPTO_MAE_ADOPTED (default off).
- Losing-coin time limit: ~24h behind TREZO_CRYPTO_TIME_EXIT (default off); 7-day outright close.
- Re-scoring: TREZO_CRYPTO_REEVAL=1 re-scores losing coins so weak ones rotate out and free pocket slots. Ships default-off; without it a pocket full of losers stays locked (this is exactly the Sep-2026 capacity lock).
- Liquidation circuit: at most one close attempt per book per symbol per 2 min; 3 consecutive failures back off for 30 min and alert once.

4.7 Options / wheel

```
CSP collateral = strike x 100 x contracts     # full cash-secured, always
covered-call risk proxy = strike x 100 x contracts x 0.1
sell premium ONLY when implied vol > realized vol (variance premium RICH)
```

- Wheel caps: max open CSPs per posture (balanced posture: 2); DTE cap via trezo_wheel_max_dte; collateral is reserved in full before the trade is even proposed — stricter than any margin model, deliberately.
- Variance premium check: selling when implied <= realized is taking the wrong side of the volatility trade; the scanner vetoes it and says both numbers.
- OCC symbology: strike encodes as an 8-digit chunk = strike x 1000; the positions endpoint and orders endpoint spell option and crypto symbols differently (Section 5).

4.8 Costs, liquidity, crowding

- Crypto real cost = fee 0.52% round trip + measured spread (bid/ask at decision time); the model uses 0.62% flat; real_cost rows compare them, observe-only. LTC routinely exceeds the model on spread — a rebuild could make cost per-coin.
- Liquidity filter: average volume minimum (config default 250,000 shares; the extended-hours lane uses 500,000). No price data = veto, never a guess.
- Crowding: TREZO_CROWDING_MAX / _STEP throttle how many concurrent names one lane may hold.
- Daily goal banking: TREZO_DAILY_GOAL / TREZO_GOAL_MAX_PCT support banking winners at a daily income target; profit-step ladders (TREZO_PROFIT_STEP_*) take partials at steps.

4.9 Kill-switches and recovery (per book)

- Each book has independent kill-switch states driven by broker-reject counters and health checks; a halted book refuses new entries while exits continue to be managed.

- Weekly RECOVERY: a recovering book trades with a raised TCS bar (recovery_bump) until it earns back normal thresholds. Ghost positions reconcile and reset reject counters (halt_cleared).
- A dead-man switch (DB migration 0052) lets the platform notice an engine that stopped ticking entirely.

5. The safety core — build these rules FIRST

Every rule here exists because a real incident booked real (paper) losses or hid real risk. They are what make the platform 'not misunderstandable'. In a clean rebuild these come before any strategy code.

- **S1. A failed read is None, never [].** 'Flat account' and 'the request failed' must be different types. One rate-limited positions read that returned [] made every position look gone; the monitor phantom-closed the whole 75k book and booked ~-$5.8k that never happened. Strict reads return None; callers treat None as 'could not check, do not act'; a failure is never cached.
- **S2. Every broker read is bound to the book it is about, at the moment it is made.** One module (book_scope) is the single door: you cannot get a held-symbol set without naming the book, and naming it binds it. Per-book cache (45s TTL), never global. Asking 'which book?' in two places that can disagree closed nine real positions per book once.
- **S3. The venue's spelling and the venue's numbers win.** Positions endpoint wants DOTUSD; orders want DOT/USD. Crypto stop quantities come from the venue's own position record (9 dp), not the ledger's 8-dp rounding — a rounded-UP qty 403s 'insufficient balance' and the coin rides with no floor. On such a 403, parse the venue's 'available: N' and retry ONCE at N; never retry the same number twice.
- **S4. Adopted positions keep the broker's clock.** When the reconciler adopts a position the executor missed, entry_at becomes the EARLIEST fill of the order that built it — but only from evidence inside the lookback window (72h); any timestamp outside [window_start, now] is refused and the row keeps now() with a spoken reason. A wrong entry_at silently moves an exit; a late one only makes a position look young.
- **S5. A swallowed exception is invisible.** The bus router announces every handler failure: bus event + activity row + one webhook ping per (agent, error). Before this net, a crash in the risk manager's message path traded NOTHING for four days while every health check said 'ticking fine'.
- **S6. Count the flow, per lane.** Signals in with zero approvals out (approval starvation), or approvals in with zero fills out (execution starvation), during that lane's market hours, is an alarm on its own — whatever the cause. Deliberate refusals count as outcomes; a window whose approvals were ALL refused on purpose is a CAPACITY LOCK (warn), not a malfunction (urgent). Crypto is judged 24/7; stock lanes only while the market is open.
- **S7. Mutations verify the route.** Before anything that moves money, assert the bound account owns the book (route guard); a mismatch raises instead of quietly hitting the default account.

6. Alerting

One env var: TREZO_ALERT_WEBHOOK (Discord or Slack webhook URL, auto-detected). Unset = silent no-op. The notify path never raises and never blocks a tick. Alerts are deduped by key with per-severity quiet times: urgent repeats every 30 min while the condition holds, warn every 4h, info once a day. Every send or failure also writes an activity row (alert_sent / alert_failed) so the local record is complete when the channel is down. A send_test proves the channel end-to-end (the relay can trigger it remotely).

7. Operations design

- **Ops relay:** a Supabase mailbox (ops_tasks) the engine drains every 5-minute tick. Exactly six CHECK-constrained job kinds: git_pull_restart, pip_install, report_status, restart_service, tail_log, web_rebuild. It is NOT a shell and can never trade. The reverse channel (ops_log_tail) pushes the activity log to Supabase so it is readable from anywhere.
- **The beacon rule:** a deploy is DONE when a NEW process says hello — engine_boot names pid, commit, and agent count. A job row saying 'done' only means the pull happened; read the result body for 'ROLLED BACK'.

- **The deploy gate:** agents/tests/run_all.py imports every tests/test_*.py and runs bare test_ functions in ONE process — no pytest, no fixtures, no .env, no network. A red gate rolls the checkout back automatically. Suites stub config, patch-and-RESTORE module attrs, and must never depend on the wall clock or the calendar (lesson L6).
- **Archives:** hourly zip of state to Supabase storage (and Dropbox weekly tier), db janitor purges agent_messages older than 48h.

8. What you need to rebuild it

8.1 Accounts and keys (obtain these; names only, values never written down here)

Service	Keys / values you need	Used for
Supabase project	SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY	All durable state. Free tier is fine to start.
Alpaca (paper)	ALPACA_API_KEY, ALPACA_SECRET_KEY, ALPACA_BASE_URL (paper-api.alpaca.markets), ALPACA_EXPECTED_ACCOU	Broker. One login can hold several paper accounts - your books.
Discord (or Slack)	TREZO_ALERT_WEBHOOK	The alert channel. Two clicks to create on a Discord server.
Anthropic	ANTHROPIC_API_KEY	Research / advisor agents.
Market data (optional)	FINNHUB_API_KEY, TWELVE_DATA_API_KEY, ALPHA_VANTAGE_API_KEY, NASDAQ_DATA_LINK_API_KEY, MASSIVE_	Enrichment: fundamentals, FX, candles, dividends. The core trades on
Dropbox (optional)	TREZO_DROPBOX_APP_KEY / _APP_SECRET / _REFRESH_TOKEN	Weekly archive tier.
Upstash Redis (optional)	UPSTASH_REDIS_REST_URL / _TOKEN	Cache tier the web app can use.
Mem0 (optional)	MEM0_API_KEY	Agent memory experiments.
Crypto exchange (optional)	KRAKEN_API_KEY / _PRIVATE_KEY or COINBASE equivalents	Only if you route crypto off-Alpaca; default is Alpaca crypto.

Also generate fresh yourself: FERNET_ENCRYPTION_KEY (any Fernet key) and CRON_SECRET (any random string). IMPORTANT: mint NEW keys for the rebuild rather than reusing the current ones - the old stack keeps running on its own keys, the new stack on its own, and nothing can cross-trade. (The one-time rotation of the current Alpaca+Supabase keys is still outstanding from the USB incident - doing the rebuild on fresh keys settles it for the new stack.)

8.2 The environment file (agents/.env)

Everything above goes in agents/.env (dotenv, loaded at boot; the Settings object in app/config.py is the one reader). Behavior toggles are all TREZO_*-prefixed with safe defaults; the ones you will actually set on day one:

```
TRADING_MODE=paper          ENV=development         PORT=8001
TREZO_PRIMARY_USER_ID=<uuid of your primary book row>
TREZO_ALERT_WEBHOOK=<your webhook>
TREZO_CRYPTO_REEVAL=1       # losing coins re-score; leave pockets unlockable
TREZO_DOC_DIR=<path to the agents' document library folder>
```

The full inventory of ~68 TREZO_* toggles is greppable in one line: grep -rhoP 'os.getenv\(\"[A-Z0-9_]+' agents/app | sort -u. Every one has a default; you do not need to set them to boot.

8.3 Database

- Run db/migrations 0001 through 0060 in order against the Supabase project (0058 adds schema_migrations bookkeeping). They create the ledger, bot_settings (per-book Bot Tuning: min_reward_risk, tcs_threshold, max_position_pct, max_open_positions, account_posture, budgets), accounts/owner split, activity log, ops_tasks (with the six-kind CHECK), ops_log_tail, briefings, RLS policies, and the dead-man switch.
- Seed rows: one owner, one account row per book (key, account_id, label, Alpaca account number), one bot_settings row per book. TREZO_PRIMARY_USER_ID points at the primary book.

8.4 The agents' document library

The agents read a drop-box folder of documents (strategy notes, books, reports) at TREZO_DOC_DIR — this is the 'library' that is online for the agents today. It is plain files on the server's disk (per-file cap 40 MB; oversized or non-text files are skipped with a spoken library_unreadable row). To keep the library in a rebuild: copy the folder, point TREZO_DOC_DIR at it. Nothing else to migrate.

8.5 Bring-up order (do it in this order and each step proves the last)

```
1. Supabase project + run migrations + seed one book
```

```
2. agents/.env with Supabase + Alpaca paper keys (ONE book first)
3. python -m app  (or the service wrapper)  -> watch for engine_boot: 30 agents
4. relay check: queue report_status        -> every agent ticking
5. alert channel: send_test                -> {ok: true} in Discord
6. paper-trade one lane small (coverage mode) for a day; read the vetoes -
   they should all be SPOKEN reasons, never silence
7. add the other books; verify per-book binding (each book's positions
   read shows ONLY its own holdings)
8. gate: python agents/tests/run_all.py must be green before any deploy
9. new web app last - the engine never depends on it
```

9. Lessons to design in (and noise to design out)

These are the incidents the current code carries scars from. The rebuild gets to have the scar tissue without the wound.

- **L1. Built-but-not-bound is the house failure mode.** Four separate times a correct per-book value existed and a call site read the ambient/global one instead (risk_pct, kill-switch counters, reconcile binding, the R:R floor). Design: there is no ambient settings getter; every read takes user_id, enforced by grep-able convention and guard tests.
- **L2. Test the message PATH, not the policy function.** The four-day outage shipped with green tests that pinned policy as a pure function and never ran a signal through on_message. Every agent change needs one test that exercises the real handler.
- **L3. A green deploy is not a working fix.** Watch the live log for the symptom to STOP. One fix deployed green and changed nothing (wrong endpoint spelling, error swallowed).
- **L4. Every deliberate refusal must be audible** (activity row with book + numbers), and the flow watchdog must count refusals as outcomes, or it cries starvation at a healthy full book (Labor Day weekend of false urgent pings).
- **L5. Resting stops vs re-entries collide at Alpaca crypto:** a new order on a coin with a resting stop trips 'potential wash trade' 403s. Design the executor to cancel-replace around entries (or skip adds on stop-protected coins) from day one; today this is still an open item.
- **L6. Gate suites must not decay:** no wall-clock windows over fixed fixtures, no time-of-day-dependent assertions (a suite went red at 3:45 PM ET every day), no pytest fixtures the bare runner lacks, and always restore patched module attrs.
- **L7. Timezones:** the 3:45 PM rule is hardcoded 19:45Z and the report crons are UTC on EDT offsets - all drift an hour when EST returns in November. A rebuild should compute from America/New_York.
- **L8. Capacity churn:** the gate happily approves the same six names every five minutes for the executor to refuse at full pockets. A rebuild can consult capacity BEFORE approving (or cool down re-approvals of a just-refused name).
- **L9. Phantom P&L residue:** once a phantom close is proven (trade QA reconciles broker receipts vs ledger), the realized counters should be corrected in the same motion; today ~-$5.8k of never-happened DOT losses still sit in the counters.

10. What you need to DO right now

- **Nothing, to keep the current platform running** - it runs from your GitHub repo + server as-is, untouched by the rebuild. Build the new stack beside it on fresh keys (8.1) and nothing can interfere with the live books.
- Get the four non-optional credentials in 8.1 (Supabase, Alpaca paper, a webhook, Anthropic), then follow 8.5 top to bottom.
- Copy the library folder if you want the agents reading the same documents (8.4).

- When the new stack's gate is green and its beacon says hello, decide which engine owns which Alpaca account - never both on one (Section 1).